

Universidad de San Carlos de Guatemala
Facultad de Ingeniería
Escuela de Ciencias y Sistemas
Área de desarrollo de software
Software Avanzado

Lic. Marco Tulio Aldana Prillwitz
Xhunik Nikol Miguel Mutzutz



Practica 2

Índice

1. Resumen	4
2. Objetivos	4
2.1 Objetivo General	4
2.2 Objetivos Específicos	4
3. Enunciado	5
3.1 Definiciones	5
3.2 Arquitectura requerida (Microservicios + Gateway + Frontend)	5
3.2.1 Frontend (Vite + TypeScript) — obligatorio	5
3.2.2 API Gateway (NestJS, REST) — obligatorio	5
3.2.3 Orders Service (NestJS, gRPC) — obligatorio	6
3.2.4 Pricing Service (NestJS, gRPC) — obligatorio	6
3.2.5 Receipt Service (NestJS, gRPC) — obligatorio	6
3.2.6 Diagrama (obligatorio en README)	6
3.3 Modelo de datos mínimo (reglas de validez incluidas)	6
3.3.1 Orden	6
3.3.2 Paquete	7
3.3.3 Regla de validación del seguro (obligatoria)	7
3.4 Reglas de cálculo (obligatorias y exactas)	7
3.5 Contratos (OpenAPI/JSON Schema + gRPC)	8
3.5.1 REST (Gateway) — OpenAPI obligatorio	8
3.5.2 gRPC interno — Protobufs obligatorios	9
3.6 Frontend (Vite + TypeScript)	9
3.6.1 Funcionalidades mínimas obligatorias (UI)	9
3.6.2 Restricciones del frontend (obligatorias)	9
3.7 Docker y Docker Compose (obligatorio)	10
3.7.1 Dockerfiles (obligatorio)	10
3.7.2 docker-compose.yml (obligatorio)	10
3.8 CI en GitHub Actions (obligatorio)	11
3.8.1 Triggers obligatorios	11
3.8.2 Jobs mínimos obligatorios	11
3.8.3 Reglas de tagging (obligatorio, sin ambigüedad)	11
3.9 Pruebas mínimas (unitarias + contrato + build)	12
3.9.1 Unit tests (obligatorio)	12
3.9.2 Contratos (obligatorio en CI)	12
3.9.3 Frontend build (obligatorio en CI)	12
3.10 Manejo de errores (REST + gRPC)	12
3.10.1 REST (Gateway) — mapeos mínimos	12
3.10.2 gRPC interno — códigos mínimos	12
3.11 Resiliencia e idempotencia (obligatorio)	12
3.11.1 Resiliencia mínima (obligatoria)	12
3.11.2 Idempotencia en creación (obligatoria)	13

4. Entregables	13
5. Requerimientos mínimos	15
6. Restricciones	15
7. Consideraciones adicionales	15
7.1 Historial de desarrollo	15
7.2 Cálculo de nota individual	15
7.3 Integridad académica	16
8. Fecha y Modo de entrega	16

1. Resumen

En esta práctica desarrollarás un sistema llamado **Quetzal Ship** (mismo dominio de la Práctica 1) para **crear, consultar, listar, cancelar órdenes de envío y generar un recibo** con el desglose del cálculo tarifario, pero ahora implementado como una **arquitectura de microservicios** orquestados por un **API Gateway** y consumidos desde un **Frontend**. El objetivo principal de la práctica es evidenciar dominio de **separación SOA → microservicios, Docker Compose y CI** (GitHub Actions).

Tecnologías obligatorias (en esta práctica):

- Los **4 servicios backend** (Gateway, Orders, Pricing, Receipt) deben estar implementados en **NestJS + TypeScript**.
- Debe existir un **Frontend** implementado en **Vite + TypeScript**, que se comunica **únicamente** con el Gateway.
- La ejecución local debe ser reproducible con **Docker Compose**.
- Debe existir un **pipeline de CI** que compile, ejecute pruebas, valide contratos, construya imágenes y publique imágenes según reglas de tagging.

2. Objetivos

2.1 Objetivo General

Construir un sistema distribuido (microservicios) para QuetzalShip, implementado con NestJS + TypeScript, consumido por un frontend Vite + TypeScript, y verificado mediante Docker Compose y un pipeline de CI reproducible.

2.2 Objetivos Específicos

1. Definir fronteras claras por responsabilidad (no “microservicios por capas”), separando: orquestación (Gateway), órdenes (Orders), cálculo (Pricing) y recibos (Receipt).
2. Implementar contratos claros: **REST (OpenAPI/JSON Schema) en el Gateway y gRPC (.proto) para comunicación interna**.
3. Contenerizar todos los componentes (incluyendo frontend) y levantar el sistema completo en un solo comando con Docker Compose.
4. Implementar un CI en GitHub Actions que verifique calidad y funcionamiento: build, tests, validación de contratos, build/push de imágenes con tags.
5. Implementar manejo de errores consistente (REST + gRPC), resiliencia mínima (timeouts/retries controlados) e idempotencia en creación de órdenes.

3. Enunciado

3.1 Definiciones

Término	Definición
Zona	METRO, INTERIOR, FRONTERA
Tipo de servicio	STANDARD, EXPRESS, SAME_DAY
Estado	ACTIVE, CANCELLED
Orden	Conjunto de paquetes con origen, destino y tipo de servicio común, más descuentos/opciones
Paquete	Unidad física enviada dentro de una orden
Desglose	Valores intermedios del cálculo: pesos, subtotales, recargos, descuento y total

3.2 Arquitectura requerida (Microservicios + Gateway + Frontend)

Debes implementar **5 componentes en ejecución**, cada uno en su **propio contenedor**:

3.2.1 Frontend (Vite + TypeScript) — obligatorio

- SPA creada con **Vite + TypeScript**.
- Se comunica **solo** con el Gateway vía HTTP/REST.
- Debe permitir ejecutar los flujos mínimos de negocio (ver 3.6).

3.2.2 API Gateway (NestJS, REST) — obligatorio

- Implementado en **NestJS + TypeScript**.
- Expone REST y publica **OpenAPI**.
- Traduce/orquesta llamadas hacia microservicios internos por **GRPC**.
- Es el **único backend** consumido por el frontend.

3.2.3 Orders Service (NestJS, gRPC) — obligatorio

- Implementado en **NestJS + TypeScript**.
- Administra el ciclo de vida de órdenes (crear, consultar, listar, cancelar).
- Guarda órdenes **en memoria** (reiniciar borrando datos).
- En creación de orden, **debe llamar a Pricing Service** para obtener el cálculo.

3.2.4 Pricing Service (NestJS, gRPC) — obligatorio

- Implementado en **NestJS + TypeScript**.
- Único responsable del cálculo tarifario (ver 3.4).
- Retorna desglose completo y determinista.

3.2.5 Receipt Service (NestJS, gRPC) — obligatorio

- Implementado en **NestJS + TypeScript**.
- Genera recibo estable (detalle + desglose) de una orden.

Regla de frontera (obligatoria):

- Frontend → **solo** Gateway (REST).
- Gateway → microservicios internos por **GRPC**.
- No se permite consumo REST entre microservicios.
- No se permite que el frontend consuma directamente Orders/Pricing/Receipt.

3.2.6 Diagrama (obligatorio en README)

Debes incluir un diagrama (Mermaid o imagen) con:

- Componentes (5), puertos, qué se expone al host y qué queda interno.
- Flujos: Frontend→Gateway→Orders→Pricing y Frontend→Gateway→Receipt (directo o vía Orders, según definas).

3.3 Modelo de datos mínimo (reglas de validez incluidas)

El modelo de datos base debe respetar el de QuetzalShip (Práctica 1).

3.3.1 Orden

Campos mínimos:

- **orderId**: string único generado por el servidor.
- **createdAt**: timestamp generado por el servidor.
- **originZone**: Zona.
- **destinationZone**: Zona.
- **serviceType**: Tipo de servicio.
- **packages**: lista **no vacía** (1 o más).

- `discount` (opcional):
 - `type`: NONE | PERCENT | FIXED
 - `value`: número (interpretación depende del tipo)
- `insuranceEnabled`: boolean.
- `status`: Estado.
- `breakdown`: desglose calculado y almacenado (ver 3.4).
- `total`: total final calculado y almacenado.

3.3.2 Paquete

Campos mínimos:

- `weightKg`: > 0.
- `heightCm, widthCm, lengthCm`: > 0.
- `fragile`: boolean.
- `declaredValueQ`: >= 0.

3.3.3 Regla de validación del seguro (obligatoria)

Si `insuranceEnabled = true`, entonces `sum(declaredValueQ)` de todos los paquetes debe ser > 0; si no se cumple, la solicitud es inválida.

3.4 Reglas de cálculo (obligatorias y exactas)

El cálculo debe ser determinista y producir siempre el mismo resultado con la misma entrada.

Estas reglas deben implementarse en el Pricing Service y el resultado final (desglosado y total) debe almacenarse en Orders.

1. Peso volumétrico por paquete:

$$\text{volumetricKg} = (\text{heightCm} * \text{widthCm} * \text{lengthCm}) / 5000$$
2. Peso tarifable por paquete:

$$\text{billableKg} = \max(\text{weightKg}, \text{volumetricKg})$$
3. Peso tarifable total de la orden:

$$\text{orderBillableKg} = \text{suma}(\text{billableKg})$$
4. Tarifa base por kg (según zona destino):
 - METRO: 8 Q/kg
 - INTERIOR: 12 Q/kg
 - FRONTERA: 16 Q/kg
5. Subtotal base:

$$\text{baseSubtotal} = \text{orderBillableKg} * \text{rate}(\text{destinationZone})$$
6. Multiplicador por tipo de servicio:
 - STANDARD: 1.00
 - EXPRESS: 1.35
 - SAME_DAY: 1.80
$$\text{serviceSubtotal} = \text{baseSubtotal} * \text{multiplier}(\text{serviceType})$$

7. Recargos:
 - Frágil: 7 quetzales por cada paquete con `fragile = true`.
 - Seguro: si `insuranceEnabled = true`, recargo = $0.025 * \text{sum}(\text{declaredValueQ})$.
`subtotalWithSurcharges = serviceSubtotal + fragileSurcharge + insuranceSurcharge`
8. Descuento (aplica sobre `subtotalWithSurcharges`):
 - PERCENT: descuento = $\text{value}/100 * \text{subtotalWithSurcharges}$, con tope **value ≤ 35**. Si `value > 35`, inválido.
 - FIXED: descuento = `value` quetzales. El total no puede ser negativo: si `value` excede el subtotal, el total final queda en 0.00.
9. Redondeo:

El total final debe redondearse a 2 decimales. Se recomienda representar dinero internamente en centavos (enteros) o documentar la estrategia en README.

3.5 Contratos (OpenAPI/JSON Schema + gRPC)

3.5.1 REST (Gateway) — OpenAPI obligatorio

Debe existir un contrato OpenAPI en:

- `contracts/openapi/quetzalship-gateway.yaml` (o `.json`)

El Gateway debe exponer como mínimo:

1. `POST /v1/orders`
 - Crea una orden.
 - Debe soportar idempotencia vía header `Idempotency-Key` (ver 3.11).
 - Respuesta mínima: `orderId`, `status`, `createdAt`, `breakdown`, `total`.
2. `GET /v1/orders`
 - Lista las órdenes en resumen.
 - Debe incluir ACTIVE y CANCELLED.
 - Si implementas paginación: defaults explícitos y límites máximos (ej. `pageSize` default 20, máximo 100).
3. `GET /v1/orders/{orderId}`
 - Obtiene detalle completo: orden completa + desglose.
4. `POST /v1/orders/{orderId}/cancel`
 - Cancela una orden (ver 3.10 y 3.11 para errores/idempotencia de cancelación).
5. `GET /v1/orders/{orderId}/receipt`
 - Genera recibo estable. Debe corresponder exactamente a lo calculado/almacenado para esa orden.

Validación obligatoria: el Gateway debe validar parámetros y body (schema) y devolver 400 con detalle cuando sea inválido.

3.5.2 gRPC interno — Protobufs obligatorios

Debe existir un directorio:

- `contracts/proto/`

Archivos mínimos:

- `contracts/proto/orders.proto`
- `contracts/proto/pricing.proto`
- `contracts/proto/receipt.proto`

Requisitos:

- Los `.proto` deben compilar en CI (ver 3.8).
- Las operaciones deben estar alineadas al comportamiento solicitado (crear/listar/detalle/cancelar/recibo), aunque estén distribuidas entre servicios.

3.6 Frontend (Vite + TypeScript)

3.6.1 Funcionalidades mínimas obligatorias (UI)

El frontend debe incluir pantallas/componentes para ejecutar estos flujos de extremo a extremo (sin usar Postman):

1. **Crear orden**
 - Formulario que permita ingresar: zonas, tipo de servicio, lista de paquetes, discount (opcional), insuranceEnabled.
 - Debe mostrar: `orderId`, `total` y un resumen del desglose.
2. **Listar órdenes**
 - Tabla/lista que muestre: `orderId`, destino, tipo servicio, estado, total.
3. **Ver detalle de orden**
 - Vista con todos los campos y desglose.
4. **Cancelar orden**
 - Acción desde detalle o lista. Debe reflejar el cambio de estado.
5. **Ver recibo**
 - Vista que consuma `/receipt` y muestre detalle + desglose (recibo estable).

3.6.2 Restricciones del frontend (obligatorias)

- Debe ser **Vite + TypeScript**.
- Solo se admiten librerías/frameworks **compatibles con Vite** (es decir, que funcionen con el flujo estándar de Vite sin migrar a otro bundler).
- El frontend no puede consumir microservicios internos: solo Gateway.
- Debe existir configuración por variable de entorno, por ejemplo:
 - `VITE_API_BASE_URL=http://localhost:3000`

3.7 Docker y Docker Compose (obligatorio)

3.7.1 Dockerfiles (obligatorio)

Debe existir un `Dockerfile` por componente:

- `services/frontend/Dockerfile`
- `services/gateway/Dockerfile`
- `services/orders/Dockerfile`
- `services/pricing/Dockerfile`
- `services/receipt/Dockerfile`

Requisitos mínimos por Dockerfile:

- Multi-stage recomendado (build + runtime).
- Variables de entorno documentadas.
- Puertos expuestos correctamente.

3.7.2 docker-compose.yml (obligatorio)

Debe existir un `docker-compose.yml` (o `compose.yml`) en raíz que:

- Levante los **5 contenedores**.
- Exponga al host **solo**:
 - Frontend (ej. 5173 o 80, según cómo lo sirvas)
 - Gateway (ej. 3000)
- Mantenga Orders/Pricing/Receipt en red interna (sin puertos al host), salvo que documenten “debug” claramente.

Health Checks obligatorios:

- Cada servicio debe tener healthcheck verificable:
 - Gateway: endpoint HTTP `/health`.
 - Microservicios gRPC: puede ser `/health` HTTP interno o gRPC health.
 - Frontend: health check HTTP (sirviendo app) si lo expones como servidor web.

Comando de ejecución obligatorio (local):

- `docker compose up --build`
- Debe quedar documentado en README y debe funcionar sin pasos manuales ocultos.

3.8 CI en GitHub Actions (obligatorio)

Debe existir un workflow:

- `.github/workflows/ci.yml`

3.8.1 Triggers obligatorios

- `pull request` hacia `main`
- `push` a `main`
- `push` a `release/**`

3.8.2 Jobs mínimos obligatorios

1. **quality**
Debe ejecutar (como mínimo):
 - Instalación con lock (`npm ci` o equivalente).
 - Lint (si existe).
 - Unit tests de backend (si aplica).
 - **Build del frontend** con Vite: debe fallar si no compila.
2. **contracts**
 - Validación de OpenAPI (lint/validate).
 - Compilación de `.proto` (o buf, si lo adoptas y documentación).
3. **docker-build**
 - Build de imágenes para: frontend, gateway, orders, pricing, receipt.
 - Los tags deben ser deterministas (ver 3.8.3).
4. **docker-push** (solo en push a `main` o `release/**`)
 - Login al registry (Docker Hub u otro indicado) usando **Secrets**.
 - Push de las 5 imágenes.

3.8.3 Reglas de tagging (obligatorio, sin ambigüedad)

Para **cada imagen**:

- En `push` a `main`:
 - Tag: `main-<SHORT_SHA>`
 - (Opcional) también `main-latest`
- En `push` a `release/<X>` (ej. `release/v2.0.0`):
 - Tag: `<X>` exactamente (ej. `v2.0.0`)
- En `pull request`:
 - Solo build (no push obligatorio).
 - Tag sugerido para logs: `pr-<PR_NUMBER>-<SHORT_SHA>`

El README debe listar el nombre de repositorio de imagen y ejemplos de tags.

3.9 Pruebas mínimas (unitarias + contrato + build)

3.9.1 Unit tests (obligatorio)

Debe existir un set de pruebas ejecutables en CI, con comando documentado (ej. `npm test`).

Mínimos:

- Pricing: cubrir reglas completas del cálculo (3.4), incluyendo percent tope 35 y fixed piso 0.00.
- Orders: crear almacena desglose/total; cancelar congela y no permite doble cancelación sin error controlado.
- Gateway: al menos pruebas de validación (payload inválido → 400).

3.9.2 Contratos (obligatorio en CI)

- OpenAPI válido.
- `.proto` compila.

3.9.3 Frontend build (obligatorio en CI)

- `npm run build` debe ejecutarse y pasar (fallar si hay errores TypeScript/Vite).

3.10 Manejo de errores (REST + gRPC)

3.10.1 REST (Gateway) — mapeos mínimos

- **400**: validación fallida (campo/regla específica).
- **404**: orderId no existe.
- **409**: operación inválida por estado (ej. cancelar ya cancelada).
- **503/504**: dependencia interna no disponible o timeout (documenta criterio exacto que usarás).

3.10.2 gRPC interno — códigos mínimos

- `INVALID_ARGUMENT`: validaciones (dimensiones ≤ 0 , seguro inválido, percent > 35).
- `NOT_FOUND`: orderId inexistente.
- `FAILED_PRECONDITION`: transición inválida (ej. cancelar ya cancelada).

3.11 Resiliencia e idempotencia (obligatorio)

3.11.1 Resiliencia mínima (obligatoria)

Para llamadas gRPC internas:

- Timeout por defecto documentado (ej. 2s) y configurable por env var.
- Retries máximo 2 **solo** en errores transitorios (ej. UNAVAILABLE / DEADLINE_EXCEEDED).
- Backoff simple (ej. 100ms, 200ms).
- Si Pricing falla, la creación de orden debe fallar de forma controlada y el Gateway debe mapearlo a error REST coherente.

3.11.2 Idempotencia en creación (obligatoria)

El Gateway debe aceptar header **Idempotency-Key** en **POST /v1/orders**.

Regla:

- Misma **Idempotency-Key** + mismo payload → debe retornar **la misma respuesta** (sin duplicar orden).
- Misma **Idempotency-Key** + payload distinto → debe retornar error (409 recomendado) indicando conflicto.

Implementación aceptada (simple):

- Tabla/mapa en memoria (Orders o Gateway) guardando: key → hash(payload) → orderId/response.

4. Entregables

Se entrega un repositorio por práctica en GitHub con nombre:

- **<NUM GRUPO>_SoftwareAvanzado_Practica2** (mismo patrón de P1, adaptado a P2).
Además deben agregar el usuario del auxiliar como colaborador: **xhuniktzi**.

Contenido obligatorio del repositorio (mínimos):

Entregable	Ubicación sugerida	Condición
Frontend Vite + TS	services/frontend/	funcional (consume Gateway)
Gateway NestJS (REST)	services/gateway/	funcional

Orders NestJS (gRPC)	<code>services/orders/</code>	funcional
Pricing NestJS (gRPC)	<code>services/pricing/</code>	funcional
Receipt NestJS (gRPC)	<code>services/receipt/</code>	funcional
OpenAPI	<code>contracts/openapi/</code>	válido
Protos	<code>contracts/proto/</code>	compilan
Dockerfiles	en cada servicio	build OK
Docker Compose	raíz	<code>docker compose up --build</code>
CI	<code>.github/workflows/ci.yml</code>	corre en PR/push
Unit tests	según estructura	ejecutables en CI
README	<code>README.md</code>	instalación/ejecución/pruebas/CI/tags
Integrantes	<code>INTEGRANTES.md</code>	5 integrantes: nombre/carné/rol
Tags Git	<code>P2-DELIVERY</code>	apunta al commit final de entrega

5. Requerimientos mínimos

Para tener derecho a calificación, debe cumplirse todo lo siguiente:

1. Los 4 servicios backend están implementados en NestJS + TypeScript.
2. Existe frontend Vite + TS funcional consumiendo el Gateway.
3. Gateway expone REST y valida con OpenAPI.
4. Comunicación interna por gRPC entre servicios (al menos Orders→Pricing y Gateway→Orders/Receipt).
5. Reglas de cálculo implementadas exactamente (3.4) y cubiertas por unit tests.
6. Cancelación cumple congelamiento y no elimina datos.
7. Docker Compose levanta todo el sistema con `docker compose up --build`.
8. CI corre en PR y ejecuta: quality + contracts + docker-build.
9. En push a `main` o `release/**`, CI publica imágenes con tags según 3.8.3.

6. Restricciones

- Persistencia: en memoria (reiniciar borra datos).
- Backend: estrictamente NestJS + TypeScript (los 4 servicios).
- Frontend: estrictamente Vite + TypeScript; solo librerías soportadas por Vite.
- El consumo oficial del frontend es REST vía Gateway; no consumo directo de microservicios internos.
- No se requiere autenticación/autorización.
- No se aceptan “microservicios por capas” (controller/service/repository en contenedores separados): la separación debe ser por responsabilidad de negocio.

7. Consideraciones adicionales

7.1 Historial de desarrollo

Se revisará el historial de commits para verificar avance incremental (no realizado en un único commit).

7.2 Cálculo de nota individual

La nota individual se obtiene a partir de la nota grupal y un índice de aporte calculado con Git Fame. Para efectos de este curso, **los equipos son siempre de 5**, por lo que el “aporte esperado” por integrante es **20** en una escala de 0 a 100.

Valores (0 a 100) tomados de Git Fame

- **LOC**: aporte por líneas de código
- **COMMITs**: aporte por commits
- **FILES**: aporte por archivos modificados

Cálculos

- $\text{ÍNDICE} = 0.5 * \text{LOC} + 0.2 * \text{COMMITTS} + 0.3 * \text{FILES}$
- $\text{FACTOR} = \text{MIN}(\text{ÍNDICE} / 20, 1)$
- $\text{NOTA INDIVIDUAL} = \text{NOTA GRUPAL} * (0.6 + 0.4 * \text{FACTOR})$

Campo	Fórmula
ÍNDICE	$0.5 * \text{LOC} + 0.2 * \text{COMMITTS} + 0.3 * \text{FILES}$
FACTOR	$\text{MIN}(\text{ÍNDICE} / 20, 1)$
NOTA INDIVIDUAL	$\text{NOTA GRUPAL} * (0.6 + 0.4 * \text{FACTOR})$

7.3 Integridad académica

La copia total o parcial implica calificación de 0 y será reportada a la escuela de ciencias y sistemas.

8. Fecha y Modo de entrega

- **Fecha límite:** domingo 14 de diciembre, 23:59.
- **Entrega:** enlace del repositorio GitHub en la plataforma UEDi. El repositorio debe ser accesible e incluir todos los entregables y tags requeridos.